



Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет
имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ _____ «Информатика и системы управления»

КАФЕДРА _____ «Теоретическая информатика и компьютерные технологии»

Лабораторная работа № 4
«Вычисление собственных значений и собственных векторов
симметричной матрицы методом А.М. Данилевского»
по курсу «Численные методы линейной алгебры»

Студент группы ИУ9-71Б Лысенко В. В.

Преподаватель Посевин Д. П.

Москва 2024

1 Цель работы

Реализовать метод вычисления собственных значений и собственных векторов симметричной матрицы методом А.М. Данилевского.

2 Задачи

1. Реализовать метод вычисления собственных значений и собственных векторов симметричной матрицы методом А.М. Данилевского.
2. Реализовать метод поиска собственных значений действительной симметричной матрицы A размером 4×4 .
3. Проверить корректность вычисления собственных значений по теореме Виета.
4. Проверить выполнение условий теоремы Гершгорина о принадлежности собственных значений соответствующим объединениям кругов Гершгорина.
5. Вычислить собственные вектора и проверить выполнение условия ортогональности собственных векторов.
6. Проверить решение на матрице приведенной в презентации.
7. Продемонстрировать работу приложения для произвольных симметричных матриц размером $n \times n$ с учетом выполнения пунктов приведенных выше.

3 Практическая реализация

Ниже представлен код программы:

```
using Random
using LinearAlgebra
using Symbolics
using Polynomials

eps = 1e-10

function generate_symmetric_matrix(n)
    A = rand(Float64, n, n)
    for i in 1:n
        for j in i+1:n
            A[i,j] = A[j, i]
        end
    end
    return A
end

function roots_for_Daniel(coefficients)
    x = -reverse(coefficients)
    append!(x, 1)
    p = Polynomial(x)

    return roots(p)
end

function danilevsky_method(A)
    n = size(A, 1)
    lambda_values = Vector{Float64}()
    lambda_vectors = Vector{Vector{Float64}}()
    main_B = I(n)

    B = A

    for k in 1:n-1
        B = zeros(Float64, n, n)
        for i in 1:n
            B[i,i] = 1.0
        end
        for j in 1:n
            B[n-k,j] = - A[n+1-k,j] / A[n+1-k, n-k]
            if j==n-k
                B[n-k,j] = 1 / A[n+1-k, n-k]
            end
        end
    end
end
```

```

        end
    end

    A = inv(B) * A * B

    main_B *= B
end

lambda_values = roots_for_Daniel(A[1, :])

for val in lambda_values
    my_vector = Vector{Float64}()
    for i in 0:n-1
        push!(my_vector, val^i)
    end

    my_vector = main_B * reverse(my_vector)

    push!(lambda_vectors, my_vector / norm(my_vector))
end
return lambda_values, lambda_vectors
end

function check_viete(lambda_values, A)
    trace_A = tr(A)
    det_A = det(A)
    sum_lambda_values = sum(lambda_values)
    product_lambda_values = prod(lambda_values)

    if isapprox(sum_lambda_values, trace_A) && isapprox(product_lambda_values, det_A)
        println("Проверка теоремы Виета пройдена ")
    else
        println("Проверка теоремы Виета НЕ пройдена ")
    end
end

function check_gershgorin(lambda_values, A)
    n = size(A, 1)
    is_valid = true

    for i in 1:n
        row_sum = sum(abs.(A[i, :]) - abs(A[i, i]))
        if abs(lambda_values[i] - A[i, i]) > row_sum
            is_valid = false
        end
    end
end

```

```

end

if is_valid
    println("Проверка круговГершгоринапройдена ")
else
    println("Проверка круговГершгоринаНЕпройдена ")
end
end

function are_orthogonal(vectors)
    n = size(vectors, 1)

    is_ortagonal = true
    for i in 1:n
        for j in 1:n
            if i != j
                dot_product = dot(vectors[i], vectors[j])
                if eps > dot_product
                    dot_product = 0
                end
                is_ortagonal = isapprox(dot_product, 0)
            end
            if !is_ortagonal
                break
            end
        end
        if !is_ortagonal
            break
        end
    end
end

if is_ortagonal
    println("Собственные вектораортогональны ")
else
    println("Собственные векторанеортогональны ")
end
end

A = [2.2 1 0.5 2;
      1 1.3 2 1;
      0.5 2 0.5 1.6;
      2 1 1.6 2]
A = Float64.(copy(A))
println("Симметричная матрица A:")
for i in 1:4

```

```

        for j in 1:4
            print(A[i,j], " ")
        end
        println()
    end
    println()

lambda_values, lambda_vectors = danilevsky_method(A)
println("Собственные значения:")
println(lambda_values)
println()
println("Собственные векторы:")
for vec in lambda_vectors
    println(vec)
end
println()

check_viete(lambda_values, A)
check_gershgorin(lambda_values, A)
are_orthogonal(lambda_vectors)
println()

B_size = 6
B = generate_symmetric_matrix(B_size)
println("Симметричная матрица B:")
for i in 1:B_size
    for j in 1:B_size
        print(B[i,j], " ")
    end
    println()
end
println()

lambda_values, lambda_vectors = danilevsky_method(B)
println("Собственные значения:")
println(lambda_values)
println()
println("Собственные векторы:")
for vec in lambda_vectors
    println(vec)
end
println()

check_viete(lambda_values, B)
check_gershgorin(lambda_values, B)
are_orthogonal(lambda_vectors)

```

```
|| println()
```

4 Результаты

```
Симметричная матрица A:
2.2  1.0  0.5  2.0
1.0  1.3  2.0  1.0
0.5  2.0  0.5  1.6
2.0  1.0  1.6  2.0

Собственные значения:
[-1.4200865939506204, 0.22263592713261518, 1.5454183350534154, 5.652032331764597]

Собственные векторы:
[-0.2220428365454715, 0.5159103236551118, -0.7572742312071336, 0.3332705439047436]
[-0.5219205710113893, -0.45486932161400206, 0.15344701837525604, 0.705086399217363]
[0.62892976467108, -0.5725742255591186, -0.4856537967631012, 0.20185761572390468]
[0.5317360693095103, 0.4461941219087124, 0.4088155341850386, 0.5924841071103819]

Проверка теоремы Виета пройдена
Проверка кругов Гершгорина НЕ пройдена
Собственные вектора ортогональны

Симметричная матрица B:
0.5957921456704688  0.6753491835465291  0.125354079242463  0.1981543471163426  0.7098985804068917  0.7646481269223233
0.6753491835465291  0.4576909314823422  0.04041265291694618  0.4816711865928366  0.5283840410215503  0.933718754283413
0.125354079242463  0.04041265291694618  0.33811098776510984  0.13865055106470692  0.056035347672278024  0.7820913525828651
0.1981543471163426  0.4816711865928366  0.13865055106470692  0.4220055201892512  0.21215678114433345  0.7160201750263625
0.7098985804068917  0.5283840410215503  0.056035347672278024  0.21215678114433345  0.2796034384626698  0.570053511973397
0.7646481269223233  0.933718754283413  0.7820913525828651  0.7160201750263625  0.570053511973397  0.4668185907513258

Собственные значения:
[-0.8288490982791549, -0.3054749402616923, -0.20744647255223522, 0.30559892420063356, 0.620800254935724, 2.9753929462778963]

Собственные векторы:
[-0.13432404162295886, -0.36192593314489263, -0.4583793884635881, -0.21575380342673625, -0.07172477029379448, 0.7675681472183553]
[-0.6865871190330103, 0.24803351440312887, 0.06440270984015009, -0.21283129573952514, 0.6452563967099892, 0.03573257246134111]
[-0.11019892963162249, 0.6973406702091615, -0.02683744223052499, -0.4463573639455426, -0.536389492539649, 0.11791248118604769]
[0.3362158963999043, -0.26732886825352375, 0.5155594756973486, -0.7199648539159815, 0.16858754157228434, 0.05405020052993851]
[-0.4243035084469964, -0.18314950742280064, 0.687350924498726, 0.2984066460526716, -0.3678356808652757, 0.2993691412204615]
[0.45314101495768816, 0.4649693736671818, 0.21625797674167613, 0.31862567728110164, 0.3564564557682296, 0.5505594087185988]

Проверка теоремы Виета пройдена
Проверка кругов Гершгорина пройдена
Собственные вектора ортогональны
```

5 Выводы

В ходе выполнения лабораторной работы был реализован метод А.М. Данилевского по вычислению собственных значений и собственных векторов. Также были реализованы проверки условий теоремы Гершгорина о принадлежности собственных значений соответствующим объединениям кругов Гершгорина и корректность вычисления собственных значений по теореме Виета. Метод был реализован для симметричных матриц произвольного размера. Корректность всех вычислений была проверена на матрице, приведённой в презентации.